

07_EDA_CON_PANDAS

August 21, 2026



1 MÓDULO 07: Análisis Exploratorio de Datos (EDA) y Curación de Metadatos Científicos de Digital.CSIC

Curso: Python para Ciencia Abierta (CSIC)

Autor: Gustavo Liñán Cembrano

Fechas: 5 a 8 de Octubre de 2026

Tiempo Estimado de Impartición: 1 hora 30 minutos

Módulo: 07 - Exploratory Data Analysis (EDA), OAI-PMH Scraping & Data Curated Pipelines

1.1 1. Objetivos de Aprendizaje

- **Ingesta de Datos en Repositorios Abiertos:** Muestrear y descargar metadatos reales desde el protocolo OAI-PMH de **Digital.CSIC** utilizando la librería `digital_csic`.
- **Cortesía de Red (Throttling):** Respetar la carga del servidor aplicando pausas deliberadas entre peticiones HTTP (`delay = 1.0s` por registro).
- **Simulación de Corrupción de Datos:** Comprender los errores típicos en bases de datos científicas mediante la inyección controlada de anómalos (fechas de 5 dígitos por multiplicación, handles inexistentes, duplicados y nulos).
- **Análisis Exploratorio de Datos (EDA):** Dominar técnicas de inspección visual y estadística de calidad de datos (`shape`, `head`, `tail`, `info`, `describe`).
- **Pipelines de Limpieza y Curación:**
 - Desduplicación por identificadores primarios (`drop_duplicates`).

- **Validación de Identificadores por Consulta al Repositorio:** Verificar la validez de los Handles comprobando la existencia real del registro en Digital.CSIC mediante su API/OAI-PMH en lugar de depender únicamente de expresiones regulares cambiantes.
- Detección y corrección de fechas fuera de rango (detección de años de más de 4 dígitos).
- Imputación y normalización de textos y metadatos incompletos.
- **Publicación de Datasets Curados:** Exportar el conjunto de datos resultante en formatos estándar de Ciencia Abierta.

1.2 2. Configuración e Importación de Librerías

Importamos los módulos necesarios para el análisis de datos (`pandas`, `numpy`), generación de números aleatorios (`random`) e integración con la librería propia `digital_csic`.

Formato de Importación Estándar: Usamos el alias `dcsic` para nuestra librería personalizada colocada en el directorio `lib/`.

```
[ ]: import sys
import os
import random
import pandas as pd
import numpy as np

[ ]: # Añadir el directorio raíz del proyecto a sys.path para acceder al paquete lib/
root_path = os.path.abspath("..") if os.path.exists("../lib") else os.path.
↳abspath(".")
if root_path not in sys.path:
    sys.path.append(root_path)

import lib.digital_csic as dcsic
from lib.mylib import linea as linea
# Ruta base flexible para el directorio DATASETS
datasets_dir = "../DATASETS" if os.path.exists("../DATASETS") else "DATASETS"
linea()
print(f"Versión de Pandas: {pd.__version__}")
print(f"Librería digital_csic cargada correctamente.")
print(f"Directorio de datos activo: {datasets_dir}")
linea()
```

1.3 3. Muestreo e Ingesta de Datos Científicos desde Digital.CSIC

Cargaremos la lista masiva de identificadores Handle desde DATASETS/FROM_DIGITALCSIC/input_handles.csv, seleccionaremos una **muestra aleatoria de 100 registros** y consultaremos el API OAI-PMH de Digital.CSIC aplicando **1 segundo de pausa entre peticiones** para no saturar los servidores del CSIC.

```
[ ]: # Cargar lista completa de handles disponibles
handles_file = os.path.join(datasets_dir, "FROM_DIGITALCSIC", "input_handles.
↳csv")
all_handles = dcsic.load_handles_from_file(handles_file)
```

```

NREGS=100
linea()
print(f"Total de Handles disponibles en input_handles.csv: {len(all_handles)}")

# Fijar semilla aleatoria para reproducibilidad pedagógica
random.seed(42)
sample_handles = random.sample(all_handles, NREGS)

print(f"Muestra de {NREGS} Handles seleccionados aleatoriamente.")
print("Primeros 5 Handles de la muestra:", sample_handles[:5])
linea()

```

```

[ ]: # Descarga programática de metadatos desde Digital.CSIC
# IMPORTANTE: Aplicamos delay=1.0 segundo para cortesía de red
print("Iniciando la descarga de metadatos OAI-PMH (con 1.0s de pausa por
↳cortesía)...")
df_raw = dcsic.fetch_records_batch(sample_handles, max_workers=1, delay=1.0,
↳show_progress=True)

linea()
print("Dimensiones del dataset descargado:", df_raw.shape)
print("Primeras filas obtenidas:")
print(df_raw[['handle', 'title', 'year', 'journal']].head())
linea()

# Guardar copia del dataset original limpio
raw_output_path = os.path.join(datasets_dir, "digital_csic_raw.csv")
df_raw.to_csv(raw_output_path, index=False)
print(f"Dataset original guardado en '{raw_output_path}'")

```

1.4 4. Inyección Controlada de Errores (Corrupción del Dataset)

En la práctica científica real, los datos frecuentemente sufren corrupciones durante la recolección, formateo o migración.

Para aprender a detectar y reparar estos fallos, inyectaremos **4 tipos de errores aleatorios** sobre una copia de nuestro dataset:

1. **Fechas Erróneas (5 dígitos):** Multiplicar 5 años aleatorios por 10 (ej. 2021 $\rightarrow$ 20210).
2. **Filas Duplicadas:** Duplicar aleatoriamente 5 registros mediante concatenación.
3. **Handles Inexistentes:** Sustituir 5 Handles por identificadores inexistentes en el servidor (ej. 10261/99999991).
4. **Resúmenes / Títulos Nulos:** Inserción de 5 valores faltantes (NaN / None).

```

[ ]: # Copia de trabajo para la corrupción de datos
df_corrupted = df_raw.copy()
np.random.seed(42)

```

```
[ ]: # 1. Corrupción de Fechas (Multiplicar 5 años por 10 provocando 5 dígitos)
indices_fecha = np.random.choice(df_corrupted.index, size=5, replace=False)
print("Índices de registros a los que corrempereemos la fecha:", indices_fecha)
linea()
for idx in indices_fecha:
    val = df_corrupted.loc[idx, 'year']
    if str(val).isdigit():
        df_corrupted.loc[idx, 'year'] = str(int(val) * 10)
print(f"1. Inyectadas 5 fechas erróneas (años de 5 dígitos) en índices:␣
↪{(indices_fecha)}")
```

```
[ ]: # 2. Inyección de Duplicados (Duplicar 5 filas aleatorias)
indices_dup = np.random.choice(df_corrupted.index, size=5, replace=False)
filas_duplicadas = df_corrupted.loc[indices_dup]
df_corrupted = pd.concat([df_corrupted, filas_duplicadas], ignore_index=True)

linea()
print(f"2. Duplicadas 5 filas aleatorias. Nuevo tamaño del dataset:␣
↪{df_corrupted.shape[0]}")
```

```
[ ]: # 3. Corrupción de Handles (Sustituir por identificadores inexistentes en␣
↪Digital.CSIC)
indices_handle = np.random.choice(df_corrupted.index, size=5, replace=False)
for i, idx in enumerate(indices_handle):
    df_corrupted.loc[idx, 'handle'] = f"10261/99999999{i}"

linea()
print(f"3. Inyectados 5 Handles inexistentes en el servidor (ej. 10261/
↪999999990) en índices: {(indices_handle)}")
```

```
[ ]: # 4. Inyección de Valores Faltantes (NaN en abstract)
indices_null = np.random.choice(df_corrupted.index, size=5, replace=False)
for idx in indices_null:
    df_corrupted.loc[idx, 'abstract'] = None

linea()
print(f"4. Inyectados 5 valores nulos en 'abstract' en índices:␣
↪{(indices_null)}")
```

```
[ ]: # 5. Inyección de Valores Faltantes (NaN en Título)
indices_null = np.random.choice(df_corrupted.index, size=5, replace=False)
for idx in indices_null:
    df_corrupted.loc[idx, 'title'] = None

linea()
print(f"5. Inyectados 5 valores nulos en 'title' en índices: {(indices_null)}")
```

```
[ ]: # Guardar el dataset corrompido en disco
corrupted_output_path = os.path.join(datasets_dir, "digital_csic_corrupted.csv")
df_corrupted.to_csv(corrupted_output_path, index=False)
print(f"Dataset corrompido guardado en '{corrupted_output_path}'")
linea()
```

1.5 5. Análisis Exploratorio de Datos (EDA) e Inspección Inicial

Comenzamos la fase de diagnóstico sobre el dataset corrompido `digital_csic_corrupted.csv` utilizando las funciones fundamentales de exploración en Pandas.

```
[ ]: # Cargar el dataset corrompido para iniciar la curación
corrupted_file = os.path.join(datasets_dir, "digital_csic_corrupted.csv")
df = pd.read_csv(corrupted_file)

linea()
print(f"1. Tamaño y forma del dataset (.shape): {df.shape[0]} filas, {df.shape[1]} columnas")
linea()
print("2. Primeras 3 filas (.head(3)):")
print(df[['handle', 'year', 'title']].head(3))
linea()
print("3. Últimas 3 filas (.tail(3)):")
print(df[['handle', 'year', 'title']].tail(3))
linea()
```

```
[ ]: # Resumen técnico (.info() y .describe())
linea()
print("Información de tipos y nulos (.info()):")
df.info()
linea()

print("Resumen estadístico de columnas (.describe()):")
print(df.describe(include='all'))
linea()
```

1.6 6. Bloque de Limpieza y Curación de Datos

Procederemos a solucionar metódicamente cada una de las anomalías detectadas en nuestro dataset científico.

1.6.1 6.1. Detección y Eliminación de Duplicados

```
[ ]: # Identificar duplicados basados en la columna 'handle'
n_duplicados = df.duplicated(subset=['handle']).sum()

linea()
print(f"Registros totales antes de deduplicar: {df.shape[0]}")
```

```
print(f"Número de registros duplicados por Handle: {n_duplicados}")

# Eliminar duplicados manteniendo la primera aparición
df_clean = df.drop_duplicates(subset=['handle'], keep='first').copy()

print(f"Registros tras eliminar duplicados: {df_clean.shape[0]}")
linea()
```

1.6.2 6.2. Validación de Handles: Verificación Real frente al Repositorio vs Expresiones Regulares

Lección Clave de Ciencia Abierta:

A menudo se utiliza la validación por **expresiones regulares (Regex)** para comprobar la sintaxis de un identificador (ej. buscar el patrón 10261/XXXXX). Sin embargo, **depender únicamente del formato es un error común en producción**: los formatos de URIs, prefijos institucionales o identificadores persistentes pueden evolucionar con el tiempo.

La forma verdaderamente robusta de validar la integridad de un Handle o DOI es **verificar la existencia real del registro consultando directamente el servicio OAI-PMH de Digital.CSIC**. Si el servidor no devuelve metadatos o indica error de registro no encontrado (`idDoesNotExist`), sabremos con certeza que el identificador es inválido o corrupto.

1.6.3 6.3 Creamos una pequeña función que devuelve si un registro existe o no en Digital.CSIC mediante consulta al OAI.PMH

```
[ ]: # Función de validación basada en verificación real con el API de Digital.CSIC
def verificar_existencia_handle(handle_str):
    # Consultamos al servidor de Digital.CSIC si el registro existe realmente
    record = dcsic.fetch_digital_csic_record(handle_str, enrich_external=False)
    return record is not None
```

```
[ ]: linea()
print("Iniciando la validación de Handles mediante consulta directa al_
↪ repositorio Digital.CSIC...")

# Evaluamos cada handle sobre el DataFrame cargado desde el CSV corrompido.
# Para ello creamos una columna "valido" que resulta de aplicar la función_
↪ verificar_existencia_handle
# a la columna handle

df_clean['valido'] = df_clean['handle'].apply(verificar_existencia_handle)
```

¿Qué hace exactamente el método `.apply()`?

El método `.apply()` en Pandas es como un bucle `for` automático, elegante y optimizado.

Su función es tomar una **** Serie de datos **** (una columna) y pasar elemento por elemento (celda

a celda) por una función de Python que tú elijas.

```
[ ]: # Filtrar handles inválidos (aquellos que el servidor rechazó)
handles_invalidos = df_clean[~df_clean['valido']]
print(f"Número de Handles no encontrados o inválidos en el servidor:␣
      ↳{len(handles_invalidos)}")
print("Handles defectuosos detectados por la consulta real:")
print(handles_invalidos[['handle', 'title']])
linea()

[ ]: # Conservar únicamente los registros validados por el servidor
df_clean = df_clean[df_clean['valido']].drop(columns=['valido']).copy()
print(f"Registros totales tras filtrar Handles inválidos: {df_clean.shape[0]}")
linea()
```

1.6.4 6.3. Detección y Corrección de Fechas Anómalas (Búsqueda de Años de Más de 4 Dígitos)

En bases de datos temporales, los años válidos constan de **4 dígitos** (entre 1000 y 9999). Cualquier valor con 5 o más dígitos (o `year_num >= 10000`) refleja una corrupción de datos.

```
[ ]: # Convertir columna year a numérico con errors='coerce'
df_clean['year_num'] = pd.to_numeric(df_clean['year'], errors='coerce')

# Detectar fechas anómalas buscando años con más de 4 dígitos (>= 10000)
mask_mas_de_4_digitos = df_clean['year_num'] >= 10000
fechas_erroneas = df_clean[mask_mas_de_4_digitos]

linea()
print(f"Número de fechas anómalas detectadas (más de 4 dígitos):␣
      ↳{len(fechas_erroneas)}")
print("Muestra de fechas erróneas de 5 dígitos encontradas:")
print(fechas_erroneas[['handle', 'year', 'year_num']])
linea()

# Corrección de fechas de 5 dígitos (división entera por 10 para restaurar el␣
  ↳año de 4 dígitos)
df_clean.loc[mask_mas_de_4_digitos, 'year_num'] = df_clean.
  ↳loc[mask_mas_de_4_digitos, 'year_num'] // 10

# Formatear como entero definitivo
df_clean['year'] = df_clean['year_num'].fillna(0).astype(int)
df_clean.drop(columns=['year_num'], inplace=True)

print("Fechas corregidas exitosamente. Resumen estadístico de 'year':")
print(f"Maxima fecha encontrada: {df_clean['year'].describe()}")
linea()
```

1.6.5 6.4. Tratamiento de Valores Faltantes (NaN) e Imputación

```
[ ]: # Conteo de valores nulos por columna
nulos_por_columna = df_clean.isnull().sum()

linea()
print("Recuento de valores nulos por columna:")
print(nulos_por_columna)
linea()
```

```
[ ]: # Asignacion de abstract nulos con valor por defecto
df_clean.fillna({
    'abstract': 'Sin abstract disponible',
    'journal': 'Digital.CSIC',
    'authors': 'Autor Desconocido'
}, inplace=True)

print("Nulos restantes tras la imputación:")
print(df_clean.isnull().sum())
linea()
```

```
[ ]: # Asignacion de título nulos con valor por defecto
df_clean.fillna({
    'title': 'Sin título disponible',
    'journal': 'Digital.CSIC',
    'authors': 'Autor Desconocido'
}, inplace=True)

print("Nulos restantes tras la imputación:")
print(df_clean.isnull().sum())
linea()
```

1.7 7. Análisis Exploratorio Final (EDA Curado)

Una vez curado el dataset, realizamos el análisis exploratorio definitivo sobre la distribución temporal y temática de la producción científica.

```
[ ]: # Distribución de publicaciones por década / año
publicaciones_por_año = df_clean[df_clean['year'] > 1900]['year'].
    ↪value_counts().sort_index()

linea()
print("Distribución de publicaciones por año (Top 10 años más recientes):")
print(publicaciones_por_año.tail(10))
linea()

# Top 5 revistas más frecuentes en la muestra
top_revistas = df_clean['journal'].value_counts().head(5)
```

```
print("Top 5 Fuentes / Revistas más frecuentes:")
print(top_revistas)
linea()
```

1.8 8. Exportación del Dataset Curado de Ciencia Abierta

Exportamos el dataset final 100% limpio y curado a un archivo CSV estructurado listo para su publicación e interoperabilidad.

```
[ ]: # Exportar a CSV con codificación UTF-8
output_curated = os.path.join(datasets_dir, "digital_csic_curated.csv")
dcsic.export_records_to_csv(df_clean, output_curated, index=False)

linea()
print(f"Dataset curado exportado exitosamente a: '{output_curated}'")
print(f"Registros totales finales: {df_clean.shape[0]}")
linea()
```

1.9 9. Ejercicios Prácticos y Autoevaluación

1.9.1 Ejercicio 1: Filtrado de Producción Reciente

Filtra el DataFrame curado `df_clean` para mostrar únicamente los registros publicados a partir del año **2020** inclusive. Muestra el número total de artículos y las columnas `handle`, `year` y `title`.

```
[ ]: # Solución Ejercicio 1
recientes = df_clean[df_clean['year'] >= 2020]

linea()
print(f"Número de artículos publicados a partir de 2020: {len(recientes)}")
print(recientes[['handle', 'year', 'title']].head())
linea()
```

1.9.2 Ejercicio 2: Búsqueda Palabras Clave en Títulos

Encuentra cuántos artículos contienen la palabra "investigación" o "estudio" en su título (insensible a mayúsculas/minúsculas).

```
[ ]: # Solución Ejercicio 2
mask_kw = df_clean['title'].str.
    ↪contains(r'investigación|estudio|research|study', case=False, na=False)
articulos_kw = df_clean[mask_kw]

linea()
print(f"Número de artículos con palabras clave científicas en el título: ↵
    ↪{len(articulos_kw)}")
print(articulos_kw[['handle', 'title']].head())
linea()
```